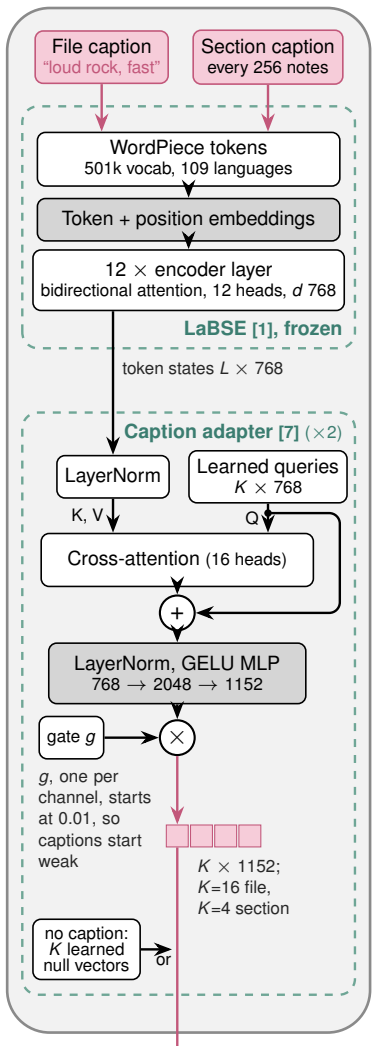
Captioned music model (477M)

477M trainable parameters: trunk 446M, cascade heads 16M, two caption adapters 13M, embeddings and length tables 2M. The LaBSE text encoder (471M) is frozen and not counted.

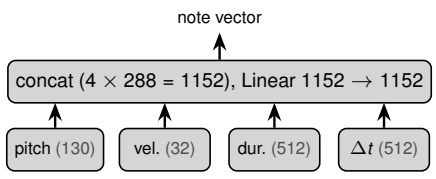
Status: trained. F1 (31,098 steps, 2.0B notes, one RTX 5090) and its 600-step caption phase (f1_cap) are done; f1_cap is released as musicar-L-caption.
Why each part is there, the training and the comparison with other open models: the second sheet, *What was found and tested*. Infilling comes in a later fine-tune.

How to read: numbers give the reading order; pink = captions; grey boxes = linear layers and embedding tables; dashed frame = a module drawn open in place; dotted frame or box = a zoom (white panel) into the dotted part.

1 Caption path



2 Input embedding



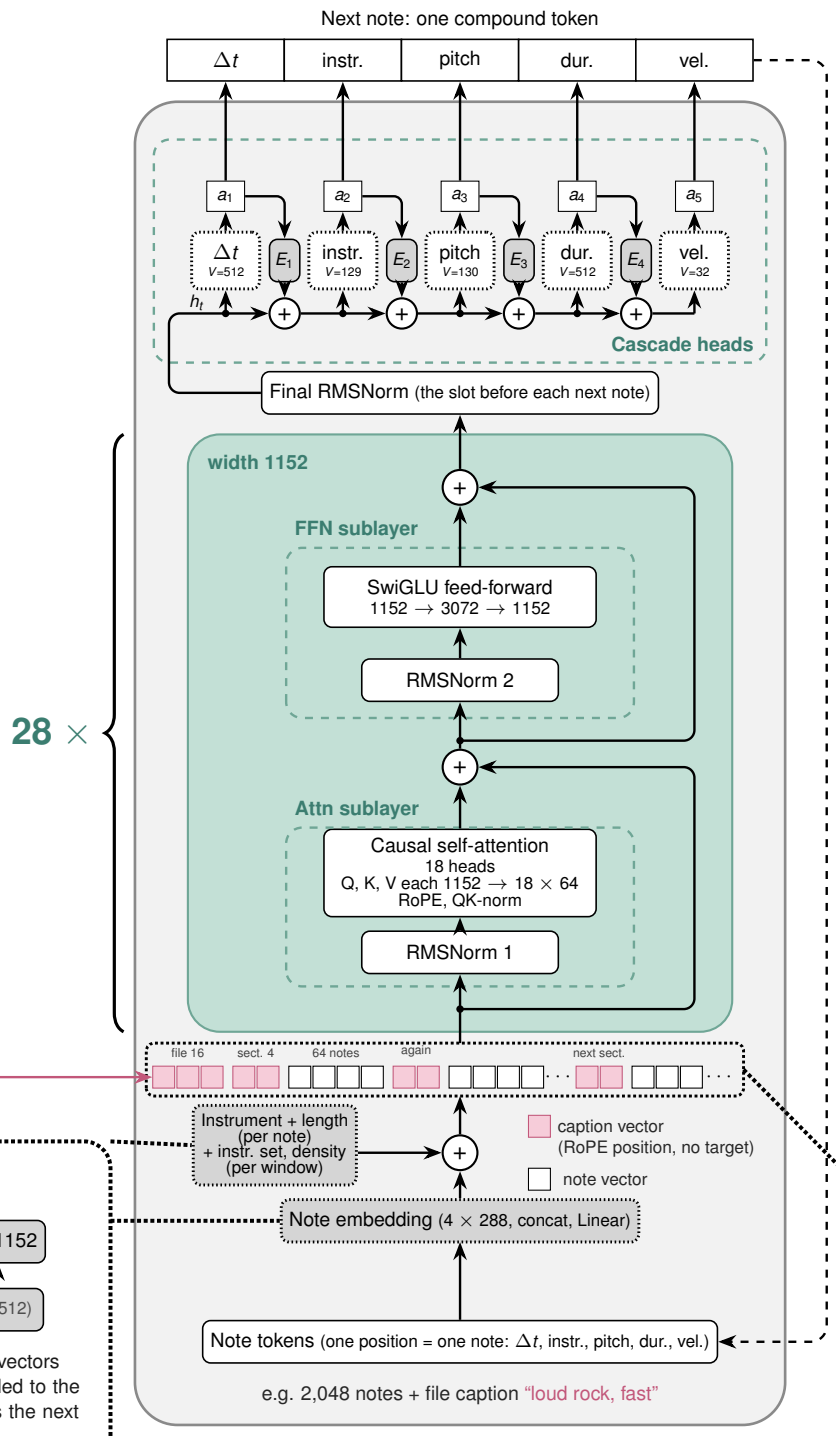
+ instrument of this note: a table of 129 vectors (128 General MIDI programs + drums), added to the note vector. The output head *instr.* predicts the next one.

+ length of the piece, two more tables per note: the **seconds left** to the end (33 levels: none, under 0.5 s, then 31 log steps of ≈28% up to 1,200 s) and the **share played** (17 levels: none + 16 equal steps). Both start at zero; in training the true length is scaled by $e^{0.1\,\varepsilon}$, $\varepsilon \sim \mathcal{N}(0,1)$ (about ±10%), so a rough request still works, and 15% of the windows get "none". In generation they come from the requested length (6).

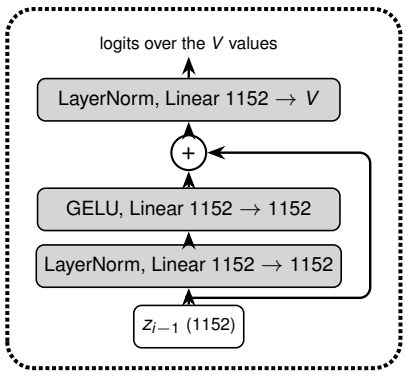
+ window conditions, the same vector at every note position: the **instrument set** of the window (129-dim multi-hot → Linear) and its **density** (notes per second in 15 levels + "none" → a table of 16 vectors). Both start at zero and are dropped 15% of the time in training, so "none" stays usable; windows with a caption get "none", so the caption is what describes them. In generation they come from the user or are measured on the prompt.

Time is absolute: Δt and durations in 20 ms steps, clipped at 10.22 s. There are no bars, beats or tempo tokens; the metre has to be read from the timing. 2,048 notes are about 1–4 minutes of music.

3 Music model



4 One cascade head (the 5 dotted boxes, own weights)



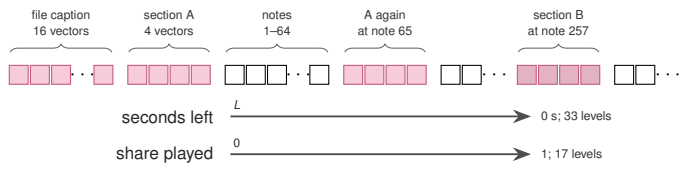
Why a cascade. One position predicts all five values of the next note. Head i reads the running state

$$z_0 = h_t, \quad z_i = z_{i-1} + E_i(a_i),$$

the trunk state h_t plus the embeddings E of the values already chosen. So the note's probability follows the chain rule, $p(\Delta t)\, p(\text{instr.} \mid \Delta t)\, p(\text{pitch} \mid \Delta t, \text{instr.}) \cdots$ (compound tokens as in [6]).

a_i is the true value in training (teacher forcing) and the sampled value in generation. Loss: the sum of the five cross-entropies. Without the chain: +0.34 nats / note at 42M (*What was found and tested*).

5 The sequence: where the caption vectors go



Positions. Every slot has one RoPE position: notes and caption vectors alike. Caption vectors have no target; the last one before a note predicts that note.

File caption first, 16 vectors in front of the notes (16 null vectors without a caption).

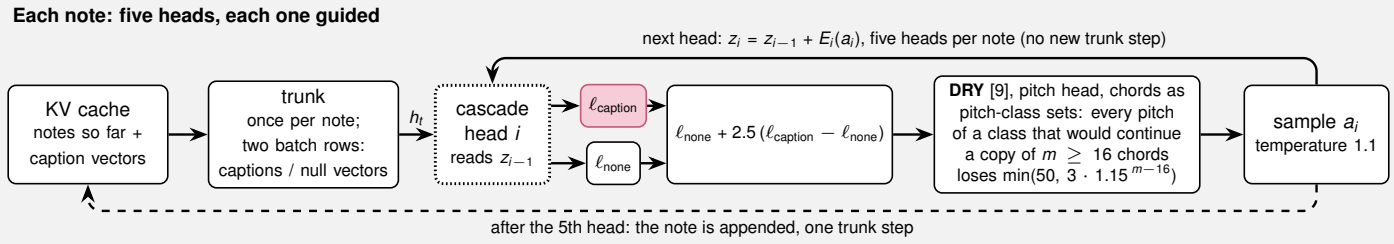
Section captions every 256 notes. A section's 4 vectors arrive just before its first note and again every 64 notes, so the caption in force is never far back. Each one describes its own section and names what changed since the one before ("drums enter, louder").

A training window: 2,048 notes + 16 + 4 per caption arrival = 2,212 positions (room for 37 section-caption arrivals = 148 vectors, fixed shape; unused ones sit after the notes and are never read). The caption phase f1_cap trains at 512 notes; generation keeps 2,048 in view.

Length levels (2) are added to the note vectors only: seconds left from L down to 0 and the share played from 0 to 1, both read from each note's onset.

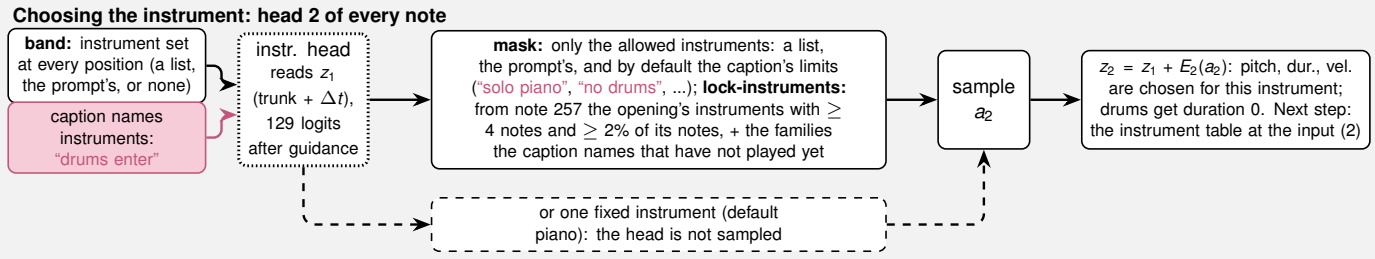
6 Generation (defaults: config/generate/f1.toml)

Each note: five heads, each one guided

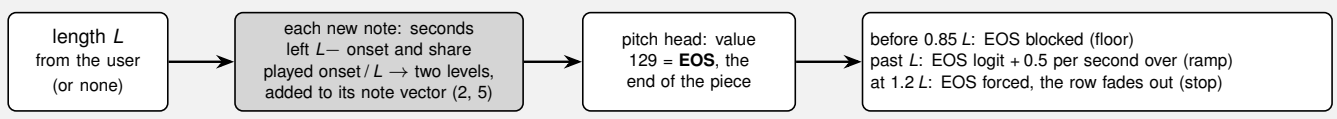


Classifier-free guidance [8]: every step runs the batch twice as wide, once with the captions and once with the null vectors; both halves are fed the same sampled note. Caption vectors enter the cache as in training (5). Past 2,048 notes the window slides by a quarter and the caption vectors are rebuilt in front of it.

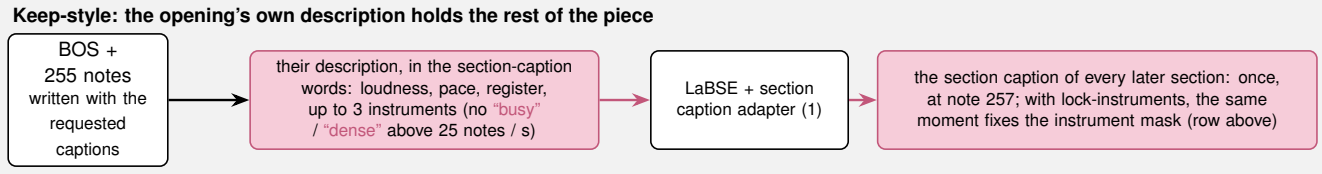
Choosing the instrument: head 2 of every note



Length and the end: the requested length L (seconds)



Keep-style: the opening's own description holds the rest of the piece



7 Glossary

Δt	time since the previous note started (20 ms steps, 512 values)	**RoPE [2]**	rotary position embedding: positions as rotations of queries and keys
instr.	instrument: General MIDI program 0–127 or drums (129 values)	**RMSNorm [3]**	layer normalisation by the root mean square only
pitch	MIDI pitch 0–127 + start / end of piece (130 values)	**SwiGLU [4]**	feed-forward layer with a SiLU-gated linear unit
dur.	note length (20 ms steps, 512 values)	**QK-norm [5]**	queries and keys normalised before the dot product
vel.	velocity: how hard the note is played (32 levels)	**LaBSE [1]**	multilingual sentence encoder (109 languages)
compound token	one sequence position = one whole note	**CFG [8]**	classifier-free guidance: push logits away from the caption-free prediction
caption vector	a caption-adapter output placed in the sequence; no target of its own, but the last vector of a section caption predicts the next note	**DRY [9]**	"don't repeat yourself" sampler: penalises continuing a passage that copies an earlier one
null vectors	learned stand-ins for "no caption"	**BOS / EOS**	start / end of piece: pitch values 128 / 129
teacher forcing	training feeds the true earlier values, not samples	**KV cache**	keys and values of earlier positions kept between steps (18 heads per layer)
multi-hot	a 0/1 vector with a 1 for every instrument present	**level**	a bin of a continuous input with its own learned vector

8 References (links)

[1] LaBSE: Language-agnostic BERT Sentence Embedding (2020), arXiv:2007.01852
[2] RoFormer: rotary position embedding (2021), arXiv:2104.09864
[3] Root Mean Square Layer Normalization (2019), arXiv:1910.07467
[4] GLU Variants Improve Transformer (2020), arXiv:2002.05202
[5] Query-Key Normalization for Transformers (2020), arXiv:2010.04245
[6] Compound Word Transformer (2021), arXiv:2101.02402
[7] Flamingo: learned-query resampler, gated adapters (2022), arXiv:2204.14198
[8] Classifier-Free Diffusion Guidance (2022), arXiv:2207.12598
[9] DRY sampler (2024), text-generation-webui PR 5677